DEEP LEARNING LAB

Algorithms for Experiments 1–8

Prepared from the uploaded DeepLearningLab.ipynb notebook. Algorithms are kept concise and aligned with the code in the notebook.

Experiment 1 — Regression

Algorithm

Step 1: Start.

Step 2: Import the required libraries and load the Advertising dataset.

Step 3: Select TV advertising as the input feature and Sales as the target for Simple Linear Regression.

Step 4: Split the data into training and testing sets.

Step 5: Train a Linear Regression model, predict Sales, and calculate MSE and R^2 score. Plot the data and regression line.

Step 6: Select TV, Radio, and Newspaper as input features for Multiple Linear Regression.

Step 7: Train the model, predict Sales, calculate MSE and R^2 score, and plot actual versus predicted Sales.

Step 8: Transform the input feature into polynomial features of degree 2.

Step 9: Train the Polynomial Regression model, predict Sales, calculate MSE and R^2 score, and plot the polynomial curve.

Step 10: Display the results and stop.

Experiment 2 — K-Nearest Neighbors (KNN) Classification

Algorithm

Step 1: Start.

Step 2: Import the required libraries and generate a two-class classification dataset.

Step 3: Split the dataset into training and testing sets.

Step 4: Create a KNN classifier with K = 5.

Step 5: Train the classifier using the training data.

Step 6: Predict the class labels of the test data.

Step 7: Calculate and display accuracy, precision, and recall.

Step 8: Stop.

Experiment 3 — Naive Bayes Classification

Algorithm

Step 1: Start.

Step 2: Import the required libraries and load the Iris dataset.

Step 3: Separate the features and target labels.

Step 4: Split the data into training and testing sets using stratified sampling.

Step 5: Create and train a Gaussian Naive Bayes classifier.

Step 6: Predict the classes of the test samples.

Step 7: Calculate and display accuracy, weighted precision, and weighted recall.

Step 8: Stop.

Experiment 4 — Decision Tree using ID3

Algorithm

Step 1: Start.

Step 2: Import the required libraries and create the Play Tennis dataset.

Step 3: Encode the categorical attributes and target labels into numerical values.

Step 4: Separate the input attributes from the target class.

Step 5: Create a Decision Tree classifier using the entropy criterion to represent the ID3 information-gain approach.

Step 6: Train the decision tree using the dataset.

Step 7: Encode a new sample using the previously fitted encoders and predict its class.

Step 8: Decode and display the predicted class.

Step 9: Plot the decision tree and stop.

Experiment 5 — Hierarchical Agglomerative Clustering

Algorithm

Step 1: Start.

Step 2: Import the required libraries and generate 150 sample points with three cluster centers.

Step 3: Compute the hierarchical linkage matrix using Ward linkage.

Step 4: Plot the dendrogram to show the order in which clusters are merged.

Step 5: Create an Agglomerative Clustering model with three clusters and Ward linkage.

Step 6: Fit the model and obtain a cluster label for each data point.

Step 7: Plot the data points using different colors for the assigned clusters.

Step 8: Display the dendrogram and clustering plot, then stop.

Experiment 6 — K-Means Clustering

Algorithm

Step 1: Start.

Step 2: Import the required libraries and generate 300 sample points with four cluster centers.

Step 3: Create a K-Means model with K = 4, K-Means++ initialization, and 10 initializations.

Step 4: Fit the model and assign each data point to a cluster.

Step 5: Obtain the cluster centroids and calculate the inertia (within-cluster sum of squares).

Step 6: Display K and the inertia value.

Step 7: Plot the clusters in different colors and mark the centroids.

Step 8: Display the plot and stop.

Experiment 7 — Artificial Neural Network using Backpropagation

Algorithm

Step 1: Start.

Step 2: Import the required libraries and generate a non-linear two-class dataset using make_moons.

Step 3: Reshape the target labels and split the data into training and testing sets.

Step 4: Initialize the weights and biases for a network with two input nodes, four hidden nodes, and one output node.

Step 5: Perform forward propagation using the sigmoid activation function for the hidden and output layers.

Step 6: Calculate the binary cross-entropy loss.

Step 7: Use backpropagation to calculate gradients for the weights and biases.

Step 8: Update the weights and biases using gradient descent.

Step 9: Repeat forward propagation, loss calculation, backpropagation, and parameter updates for 5,000 epochs.

Step 10: Predict the test labels using a threshold of 0.5 and calculate classification accuracy.

Step 11: Display the accuracy and plot the training loss and decision boundary, then stop.

Experiment 8 — Feedforward Neural Network on CIFAR-10

Algorithm

Step 1: Start.

Step 2: Import PyTorch, Torchvision, Matplotlib, and the required modules.

Step 3: Select a CUDA device if available; otherwise use the CPU.

Step 4: Load and normalize the CIFAR-10 training and testing datasets, and create data loaders with batch size 64.

Step 5: Build a feedforward neural network with input size 3,072, hidden layers of 1,024, 512, and 256 neurons, and 10 output neurons, using ReLU activations.

Step 6: Initialize the baseline model weights using Xavier initialization, with no Dropout or weight decay.

Step 7: Train the baseline model for 7 epochs using Cross-Entropy loss and the Adam optimizer with learning rate 0.001.

Step 8: Build the optimized model using Kaiming initialization, Dropout of 0.3, and L2 regularization through weight decay of 0.0001.

Step 9: Train the optimized model for 7 epochs using the same loss function and learning rate.

Step 10: Evaluate both models on the test dataset after each epoch and record their accuracies.

Step 11: Plot and compare the accuracy of the baseline and optimized models, then stop.